

ADC

Add With Carry

ADC dst, src

dst: R

ADCB

src: R

Operation: $\text{dst} \leftarrow \text{dst} + \text{src} + \text{C}$

Flags: **C:** Set if arithmetic carry from result MSB; cleared otherwise.

Z: Set if result is zero; cleared otherwise.

S: Set if result is negative; cleared otherwise.

V: Set if arithmetic overflow; cleared otherwise.

D: Cleared (ADCB);

Unaffected (ADC).

H: Set if arithmetic carry from bit 3; cleared otherwise (ADCB);

Unaffected (ADC).

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	ADC Rd, Rs	10110101_Rsss_Rddd	5
	ADCB Rbd, Rbs	10110100_Rbss_Rbdd	5

Add

dst: R

src: R, IM, IR, DA, X

src: R, IM, IR, DA, X

Operation: $\text{dst} \leq \text{dst} + \text{src}$

Flags:

- C:** Set if arithmetic carry from MSB; cleared otherwise.
- Z:** Set if result is zero; cleared otherwise.
- S:** Set if result is negative; cleared otherwise.
- V:** Set if arithmetic overflow; cleared otherwise.
- D:** Cleared (ADDB);
Unaffected (ADD).
- H:** Set if arithmetic carry from bit 3; cleared otherwise (ADDB);
Unaffected (ADD).

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	ADD Rd, Rs	10000001_Rsss_Rddd	4
	ADDB Rbd, Rbs	10000000_Rbss_Rbdd	4
	ADDL RRd, RRs	10010110_RRss_RRdd	8
IM:	ADD Rd, #data	00000001_0000_Rddd #data	7
	ADDB Rbd, #data	00000000_0000_Rbdd #data #data	7
	ADDL RRd, #data	00010110_0000_RRdd #data (high) #data (low)	14

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	ADD Rd, @Rs	00000001_Rsnz_Rddd	7
	ADDB Rbd, @Rs	00000000_Rsnz_Rbdd	7
	ADDL RRd, @Rs	00010110_Rsnz_RRdd	14
DA:	ADD Rd, address	01000001_0000_Rddd address	9
	ADDB Rbd, address	01000000_0000_Rbdd address	9
	ADDL RRd, address	01010110_0000_RRdd address	15
X:	ADD Rd, addr(Rs)	01000001_Rsnz_Rddd address	10
	ADDB Rbd, addr(Rs)	01000000_Rsnz_Rbdd address	10
	ADDL RRd, addr(Rs)	01010110_Rsnz_RRdd address	16

Logical AND

ANDB

src: R, IM, IR, DA, X

Operation: $\text{dst} \leq \text{dst} \ \& \ \text{src}$

Flags:

- C:** Unaffected.
- Z:** Set if result is zero; cleared otherwise.
- S:** Set if result is negative; cleared otherwise.
- V:** Set if result parity is even; cleared otherwise (ANDB); Unaffected (AND).
- D:** Unaffected.
- H:** Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	AND Rd, Rs	10000111_Rsss_Rddd	4
	ANDB Rbd, Rbs	10000110_Rbss_Rbdd	4
IM:	AND Rd, #data	00000111_0000_Rddd #data	7
	ANDB Rbd, #data	00000110_0000_Rbdd #data #data	7
IR:	AND Rd, @Rs	00000111_Rsnz_Rddd	7
	ANDB Rbd, @Rs	00000110_Rsnz_Rbdd	7
DA:	AND Rd, address	01000111_0000_Rddd address	9
	ANDB Rbd, address	01000110_0000_Rbdd address	9

Addressing Modes	Assembly Syntax	Encoding	Clocks
X:	AND Rd, addr(Rs)	01000111_Rsnz_Rddd	10
		address	
	ANDB Rbd, addr(Rs)	01000110_Rsnz_Rbdd	10
		address	

BIT

Bit Test Static

BIT dst, src dst: R, IR, DA, X
BITB src: IM

Operation: $Z \leq !dst[src]$

Flags: **C:** Unaffected.
 Z: Set if selected bit is zero; cleared otherwise.
 S: Unaffected.
 V: Unaffected.
 D: Unaffected.
 H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	BIT Rd, #b	10100111_Rddd_bbbb	4
	BITB Rbd, #b	10100110_Rbdd_0bbb	4
IR:	BIT @Rd, #b	00100111_Rdnz_bbbb	8
	BITB @Rd, #b	00100110_Rdnz_0bbb	8
DA:	BIT address, #b	01100111_0000_bbbb address	10
	BITB address, #b	01100110_0000_0bbb address	10
X:	BIT addr(Rd), #b	01100111_Rdnz_bbbb address	11
	BITB addr(Rd), #b	01100110_Rdnz_0bbb address	11

Notes:

1. Only bits 2-0 of the opcode are used to select the bit in the case of BITB, and bit 3 of the opcode is ignored.

Bit Test Dynamic

Operation: $Z \leftarrow \neg \text{dst}[\text{src}]$

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	BIT Rd, Rs	00100111_0000_Rsss	10
		0000_Rddd_0000_0000	
	BITB Rbd, Rs	00100110_0000_Rsss	10
		0000_Rbdd_0000_0000	

1. The Z8000 microprocessor restricts the source register to be one of R0 - R7 for BITB. This restriction does not apply to the Y8002 design. Any register may be used as the source.
2. Only bits 3-0 of the source operand are used for the bit select for BIT; only bits 2-0 of the source operand are used for the bit select for BITB.
3. Only bits 11-8 of the second word of the opcode are used. All other bits in the second word of the opcode are ignored.

Call

dst: IR, DA, X

Flags: **C:** Unaffected.
 Z: Unaffected.
 S: Unaffected.
 V: Unaffected.
 D: Unaffected.
 H: Unaffected.

Notes:

- 26

CALR

Call Relative

CALR dst

dst: RA

Operation: $SP \leq SP - 2$
 $@SP \leq PC$
 $PC \leq PC - (2 \times \text{displacement})$

Flags: **C:** Unaffected.
 Z: Unaffected.
 S: Unaffected.
 V: Unaffected.
 D: Unaffected.
 H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
RA:	CALR address	1101dddd_dddd_dddd	10

Notes:

1. The Zilog documentation incorrectly states that the displacement is *added* to the Program Counter. The Zilog Z8000 devices actually *subtract* the displacement from the Program Counter. The Y8002 design matches this behavior
2. The PC used for the address calculation is the PC of the *next* instruction.
3. The displacement is a 12-bit twos-complement number in the range -2048 to +2047. Thus the destination must be in the range -4092 to +4098 from the address of the CALR instruction.

Clear

dst: R, IR, DA, X

Operation: $\text{dst} \leq 0$

Flags: **C:** Unaffected.
 Z: Unaffected.
 S: Unaffected.
 V: Unaffected.
 D: Unaffected.
 H: Unaffected.

28

Complement

COM dst
COMB

dst: R, IR, DA, X

Operation: $\text{dst} \leq \sim \text{dst}$

Flags:

- C:** Unaffected.
- Z:** Set if result is zero; cleared otherwise.
- S:** Set if result is negative; cleared otherwise.
- V:** Set if result parity is even; cleared otherwise (COMB);
Unaffected (COM).
- D:** Unaffected.
- H:** Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	COM Rd	10001101_Rddd_0000	7
	COMB Rbd	10001100_Rbdd_0000	7
IR:	COM @Rd	00001101_Rdnz_0000	12
	COMB @Rd	00001100_Rdnz_0000	12
DA:	COM address	01001101_0000_0000 address	15
	COMB address	01001100_0000_0000 address	15
	COM addr(Rd)	01001101_Rdnz_0000 address	16
	COMB addr(Rd)	01001100_Rdnz_0000 address	16

COMFLG

Complement Flag

COMFLG flags

flag: C, Z, S, P, V

Operation: $FCW[7:4] \leftarrow FCW[7:4] \wedge inst[7:4]$

Flags:

- C:** Complemented if specified; unaffected otherwise.
- Z:** Complemented if specified; unaffected otherwise.
- S:** Complemented if specified; unaffected otherwise.
- V:** Complemented if specified; unaffected otherwise.
- D:** Unaffected.
- H:** Complemented.

Addressing Modes	Assembly Syntax	Encoding	Clocks
	COMFLG flags	10001101_CZSV_0101	7

Notes:

1. The Z8000 documentation lists the H flag as undefined, when in fact it is always complemented. The Y8002 design matches this behavior.

Compare Register

dst: R

src: R, IR, DA, X

src: R, IR, DA, X

Operation: dst - src

Flags: **C:** Set if arithmetic borrow from MSB; cleared otherwise.

Z: Set if result is zero; cleared otherwise.

S: Set if result is negative; cleared otherwise.

V: Set if arithmetic overflow; cleared otherwise.

D: Unaffected.

H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
R:	CP Rd, Rs	10001011_Rsss_Rddd	4
	CPB Rbd, Rbs	10001010_Rbss_Rbddd	4
	CPL RRd, RRs	10010000_RRss_RRddd	8
IM:	CP Rd, #data	00001011_0000_Rddd #data	7
	CPB Rbd, #data	00001010_0000_Rbddd #data #data	7
	CPL RRd, #data	00010000_0000_RRddd #data (high) #data (low)	14

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	CP Rd, @Rs	00001011_Rsnz_Rddd	7
	CPB Rbd, @Rs	00001010_Rsnz_Rbdd	7
	CPL RRd, @Rs	00010000_Rsnz_RRdd	14
DA:	CP Rd, address	01001011_0000_Rddd address	9
	CPB Rbd, address	01001010_0000_Rbdd address	9
	CPL RRd, address	01010000_0000_RRdd address	15
X:	CP Rd, addr(Rs)	01001011_Rsnz_Rddd address	10
	CPB Rbd, addr(Rs)	01001010_Rsnz_Rbdd address	10
	CPL RRd, addr(Rs)	01010000_Rsnz_RRdd address	16

Compare Immediate

dst: IR, DA, X

src: IM

Flags:

- C:** Set if arithmetic borrow from MSB; cleared otherwise.
- Z:** Set if result is zero; cleared otherwise.
- S:** Set if result is negative; cleared otherwise.
- V:** Set if arithmetic overflow; cleared otherwise.
- D:** Unaffected.
- H:** Unaffected.

33

CPD

Compare and Decrement

CPD dst, src, r, cc

dst: R

CPDB

src: IR

Operation: dst - src
 Rs <= (Word) ? Rs - 2 : Rs - 1
 r <= r - 1

Flags: **C:** Set if arithmetic borrow from MSB for compare; cleared otherwise.
 Z: Set if flags match cc after the last compare; cleared otherwise.
 S: Set if result is negative for compare; cleared otherwise.
 V: Set if r is zero after the decrement; cleared otherwise.
 D: Unaffected.
 H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	CPD Rd, @Rs, r, cc	10111011_Rsnz_1000	20
		0000_Rrrr_Rddd_cccc	
	CPDB Rd, @Rs, r, cc	10111010_Rsnz_1000	20
		0000_Rrrr_Rddd_cccc	

Notes:

1. The C, Z, S and V flags are set as usual by the compare. This flag combination is used by the cc check. The Z and V flags are subsequently set per the instruction description.
2. Because the Rd, Rs and r registers are changed by the instruction, they must be separate and non-overlapping.
3. Bits 15-12 of the second word of the opcode are ignored.

CPDR

Compare, Decrement and Repeat

CPDR dst, src, r, cc

dst: R

CPDRB

src: IR

Operation: dst - src
Rs <= (Word) ? Rs - 2 : Rs - 1
r <= r - 1
repeat until cc is true or r = 0

Flags: **C:** Set if arithmetic borrow from MSB for the last compare; cleared otherwise.
Z: Set if flags match cc after the last compare; cleared otherwise.
S: Set if result is negative for the last compare; cleared otherwise.
V: Set if r is zero after the decrement; cleared otherwise.
D: Unaffected.
H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	CPDR Rd, @Rs, r, cc	10111011_Rsnz_1100	11 + 9n
		0000_Rrrr_Rddd_cccc	
	CPDRB Rd, @Rs, r, cc	10111010_Rsnz_1100	11 + 9n
		0000_Rrrr_Rddd_cccc	

Notes:

1. The C, Z, S and V flags are set as usual by the compare. This flag combination is used by the cc check. The Z and V flags are subsequently set per the instruction description.
2. Because the Rd, Rs and r registers are changed by the instruction, they must be separate and non-overlapping.
3. Bits 15-12 of the second word of the opcode are ignored.
4. This instruction samples interrupts during each iteration. If an interrupt is pending, the instruction is stopped and the interrupt accepted. The PC saved during the interrupt acknowledge cycle in this case is the PC of the running instruction, allowing the instruction to restart after the interrupt has been serviced.

CPI

Compare and Increment

CPI dst, src, r, cc

dst: R

CPIB

src: IR

Operation: dst - src
 Rs <= (Word) ? Rs + 2 : Rs + 1
 r <= r - 1

Flags: **C:** Set if arithmetic borrow from MSB for the last compare; cleared otherwise.
 Z: Set if flags match cc after the last compare; cleared otherwise.
 S: Set if result is negative for the last compare; cleared otherwise.
 V: Set if r is zero after the decrement; cleared otherwise.
 D: Unaffected.
 H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	CPI Rd, @Rs, r, cc	10111011_Rsnz_0000	20
		0000_Rrrr_Rddd_cccc	
	CPIB Rd, @Rs, r, cc	10111010_Rsnz_0000	20
		0000_Rrrr_Rddd_cccc	

Notes:

1. The C, Z, S and V flags are set as usual by the compare. This flag combination is used by the cc check. The Z and V flags are subsequently set per the instruction description.
2. Because the Rd, Rs and r registers are changed by the instruction, they must be separate and non-overlapping.
3. Bits 15-12 of the second word of the opcode are ignored.

CPIR

Compare, Increment and Repeat

CPIR dst, src, r, cc

dst: R

CPIRB

src: IR

Operation: dst - src
Rs <= (Word) ? Rs + 2 : Rs + 1
r <= r - 1
repeat until cc is true or r = 0

Flags: **C:** Set if arithmetic borrow from MSB for the last compare; cleared otherwise.
Z: Set if flags match cc after the last compare; cleared otherwise.
S: Set if result is negative for the last compare; cleared otherwise.
V: Set if r is zero after the decrement; cleared otherwise.
D: Unaffected.
H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	CPIR Rd, @Rs, r, cc	10111011_Rsnz_0100	11 + 9n
		0000_Rrrr_Rddd_cccc	
	CPIRB Rd, @Rs, r, cc	10111010_Rsnz_0100	11 + 9n
		0000_Rrrr_Rddd_cccc	

Notes:

1. The C, Z, S and V flags are set as usual by the compare. This flag combination is used by the cc check. The Z and V flags are subsequently set per the instruction description.
2. Because the Rd, Rs and r registers are changed by the instruction, they must be separate and non-overlapping.
3. Bits 15-12 of the second word of the opcode are ignored.
4. This instruction samples interrupts during each iteration. If an interrupt is pending, the instruction is stopped and the interrupt accepted. The PC saved during the interrupt acknowledge cycle in this case is the PC of the running instruction, allowing the instruction to restart after the interrupt has been serviced.

CPSD

Compare String and Increment

CPSD dst, src, r, cc

dst: IR

CPSDB

src: IR

Operation: dst - src
Rs <= (Word) ? Rs - 2 : Rs - 1
Rd <= (Word) ? Rd - 2 : Rd - 1
r <= r - 1

Flags: **C:** Set if arithmetic borrow from MSB for the last compare; cleared otherwise.
Z: Set if flags match cc after the last compare; cleared otherwise.
S: Set if result is negative for the last compare; cleared otherwise.
V: Set if r is zero after the decrement; cleared otherwise.
D: Unaffected.
H: Unaffected.

Addressing Modes	Assembly Syntax	Encoding	Clocks
IR:	CPSD @Rd, @Rs, r, cc	10111011_Rsnz_1010	25
		0000_Rrrr_Rdnz_cccc	
	CPSDB @Rd, @Rs, r, cc	10111010_Rsnz_1010	25
		0000_Rrrr_Rdnz_cccc	

Notes:

1. The C, Z, S and V flags are set as usual by the compare. This flag combination is used by the cc check. The Z and V flags are subsequently set per the instruction description.
2. Because the Rd, Rs and r registers are changed by the instruction, they must be separate and non-overlapping.
3. Bits 15-12 of the second word of the opcode are ignored.